

хarm_ros2 на ROS 2 Kilted: сводная документация (разделы 1–16)

Окружение: Ubuntu 24.04, ROS 2 Kilted, Python 3.12, MoveIt 2.14, ros2_control 5.x, gz sim 9 (Ionic)

Workspace: `~/ros2k_ws`, исходники: `~/ros2k_ws/src/xarm_ros2` (ветка `main`) **Робот:**

UFactory xArm6 (6 DoF); режимы: fake (mock), Gazebo (физика), реальный робот **Репозитории:**

<https://github.com/TolyaDenisenko/ros2kilt> (готовый проект),

<https://github.com/TolyaDenisenko/ros2kws> (workspace с xarmgz_bringup)

Все команды проверены в реальных сессиях портирования, переноса и симуляции. Порядок разделов = порядок действий при настройке с нуля.

1. Системные зависимости

```
sudo apt update
sudo apt install -y \
    ros-kilted-ros2-control \
    ros-kilted-ros2-controllers \
    ros-kilted-moveit \
    ros-kilted-moveit-msgs \
    ros-kilted-joint-state-publisher \
    ros-kilted-joint-state-publisher-gui \
    ros-kilted-ros-gz \
    ros-kilted-gz-ros2-control
```

Примечания:

- `ros-kilted-ros2-control` даёт `controller_manager`, `controller_manager_msgs`, `hardware_interface`. Без него сборка `xarm_controller` падает на `find_package(controller_manager_msgs)`.
- `ros-kilted-ros2-controllers` даёт `joint_state_broadcaster` и `joint_trajectory_controller`.
- `ros-kilted-moveit` тянет `moveit_msgs`, `moveit-planners-ompl` и др.
- `ros-kilted-ros-gz` + `ros-kilted-gz-ros2-control` — стек новой Gazebo (раздел 16).

2. Клонирование репозитория (с нуля)

```
mkdir -p ~/ros2k_ws/src
cd ~/ros2k_ws/src
git clone https://github.com/xArm-Developer/xarm_ros2.git
cd ~/ros2k_ws
```

3. Окружение (ОБЯЗАТЕЛЬНО перед сборками/запусками)

```
source /opt/ros/kilted/setup.bash
echo $CMAKE_PREFIX_PATH | tr ':' '\n' | grep -x '/opt/ros/kilted' # должна быть строка /opt/ros/kilted
```

4. Правка №1: убрать несуществующий include

△ В готовом репозитории `ros2kilt` уже применена — пропустить.

Файл: `xarm_controller/include/xarm_controller/hardware/uf_robot_system_hardware.h`

Причина: в Kilted удалён `hardware_interface/visibility_macros.hpp`, макросы не используются.

```
sed -i '/hardware_interface\/visibility_macros.hpp/d' \
~/ros2k_ws/src/xarm_ros2/xarm_controller/include/xarm_controller/hardware/uf_robot_system_hardware.h
```

5. Stub-пакеты: обход зависимости от `xarm_gazebo`

Причина: colcon требует собранный `xarm_gazebo` / `xarm_moveit_config`, но Gazebo-пакеты несовместимы с Kilted. Stub-пакеты удовлетворяют проверки colcon/ament_index.

```

cd ~/ros2k_ws

for pkg in xarm_gazebo xarm_moveit_config; do
    mkdir -p install/$pkg/share/$pkg
    mkdir -p install/$pkg/share/ament_index/resource_index/packages
    cat > install/$pkg/share/$pkg/package.xml << EOF
<?xml version="1.0"?>
<package format="3">
    <name>$pkg</name>
    <version>0.0.0</version>
    <description>Stub package for Kilted compatibility</description>
    <maintainer email="stub@stub.com">stub</maintainer>
    <license>Apache-2.0</license>
</package>
EOF
    printf '#!/bin/sh\nAMENT_PACKAGES_PREFIX_PATH="%s"\n' "$HOME/ros2k_ws/install/$pkg" \
        > install/$pkg/share/$pkg/package.sh
    chmod +x install/$pkg/share/$pkg/package.sh
    touch install/$pkg/share/ament_index/resource_index/packages/$pkg
done

```

Проверка: `ls install/xarm_gazebo/share/ament_index/resource_index/packages/` → `xarm_gazebo`.

⚠ После `rm -rf install/ build/` stub-пакеты создать заново. Stub `xarm_moveit_config` позже заменяется настоящей сборкой (6.2).

6. Сборка

6.1 Основные пакеты

```

cd ~/ros2k_ws

source /opt/ros/kilted/setup.bash

colcon build --symlink-install \
    --cmake-args -DCMAKE_BUILD_TYPE=Release \
    --event-handlers console_direct+ \
    --packages-skip realsense_gazebo_plugin xarm_gazebo d435i_xarm_setup xarm_moveit_
config

```

Допустимые предупреждения (игнорировать): deprecated `on_init`, `.h header is obsolete`.

6.2 xarmmoveitconfig

```
colcon build --packages-up-to xarm_moveit_config --symlink-install \  
  --cmake-args -DCMAKE_BUILD_TYPE=Release \  
  --event-handlers console_direct+ \  
  --packages-skip realsense_gazebo_plugin xarm_gazebo d435i_xarm_setup
```

7. Правка №2: конфиг OMPL под формат Kilted

⚠ В готовом репозитории *ros2kilt* уже применена — пропустить.

Файл: `xarm_moveit_config/config/moveit_configs/ompl_planning.yaml` **Причина:** MoveIt 2.14 читает `planning_plugins` / `request_adapters` / `response_adapters` как `string_array`; формат Humble (строки) ронял `move_group` с `ParameterTypeException`.

```
cat > ~/ros2k_ws/src/xarm_ros2/xarm_moveit_config/config/moveit_configs/ompl_planning.y  
aml << 'EOF'  
  
planning_plugins:  
  - ompl_interface/OMPLPlanner  
  
start_state_max_bounds_error: 0.1  
jiggle_fraction: 0.05  
request_adapters:  
  - default_planning_request_adapters/ResolveConstraintFrames  
  - default_planning_request_adapters/ValidateWorkspaceBounds  
  - default_planning_request_adapters/CheckStartStateBounds  
  - default_planning_request_adapters/CheckStartStateCollision  
response_adapters:  
  - default_planning_response_adapters/AddTimeOptimalParameterization  
  - default_planning_response_adapters/ValidateSolution  
  - default_planning_response_adapters/DisplayMotionPath  
EOF  
  
colcon build --packages-select xarm_moveit_config --symlink-install
```

8. Запуск (fake-режим, симуляция без физики)

```
source /opt/ros/kilted/setup.bash  
source ~/ros2k_ws/install/setup.bash  
ros2 launch xarm_moveit_config xarm6_moveit_fake.launch.py add_gripper:=false
```

RViz откроется; инициализация ~15 с. Остановка: `kill -f "ros2 launch"; kill -f rviz2`. Не использовать `demo/demo_fake.launch.py` (без контроллеров и маркеров).

9. Проверка работоспособности (в отдельном терминале)

```
source ~/ros2k_ws/install/setup.bash
ros2 pkg list | grep xarm
ros2 node list | grep -E "move_group|controller_manager|robot_state_publisher"
ros2 topic hz /joint_states          # ~50-100 Hz
ros2 topic hz /tf
ros2 run tf2_ros tf2_echo world link1 # Translation: [0, 0, 0.267]
```

RViz: `RobotModel` → `Status: Ok`; `MotionPlanning` → `Joints` двигает модель; `Planning` → маркеры → `Plan` → `Execute`.

10. Реальный робот (НЕ проверено — осторожно!)

```
ros2 launch xarm_moveit_config xarm6_moveit_realmove.launch.py \
  robot_ip:=<IP_РОБОТА> add_gripper:=false
```

⚠ В mock-режиме с недоступным IP даёт segfault в `libxarm_cxx_sdk.so` (ожидаемо). Перед боевым запуском: `ping <IP>`, аварийная кнопка, без нагрузки.

11. Итоговый список правок относительно чистого upstream xarm_ros2

#	Файл / место	Изменение
1	<code>xarm_controller/include/.../uf_robot_system_hardware.h</code>	удалён include <code>visibility_macros.hpp</code>
2	<code>xarm_moveit_config/config/moveit_configs/ompl_planning.yaml</code>	переписан на формат Kilted (списки)
3	<code>xarm_moveit_config/launch/_robot_moveit_common.launch.py</code>	<code>use_sim_time: True</code> у <code>movegroup</code> , <code>rviz2</code> , <code>statictf</code>
4	<code>xarm_description/launch/_robot_description.launch.py</code>	<code>use_sim_time: True</code> у <code>robotstatepublisher</code>
5	<code>install/xarm_gazebo/</code> (вне src)	stub-пакет вручную
6	новый пакет <code>xarm_gz_bringup</code>	Gazebo-порт (раздел 16)

12. Известные ограничения

- Не собираются: `xarm_gazebo`, `realsense_gazebo_plugin`, `d435i_xarm_setup`.
- Deprecated-warning `on_init` остаётся — не влияет.
- После `rm -rf install/ build/` — повторить раздел 5.
- Gazebo Classic недоступна; физическая симуляция — только через раздел 16.

13. Шпаргалка

```
# окружение
source /opt/ros/kilted/setup.bash && source ~/ros2k_ws/install/setup.bash

# сборка всего
cd ~/ros2k_ws && colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release \
  --packages-skip realsense_gazebo_plugin xarm_gazebo d435i_xarm_setup

# fake-режим
ros2 launch xarm_moveit_config xarm6_moveit_fake.launch.py add_gripper:=false

# Gazebo-режим
ros2 launch xarm_gz_bringup xarm6_gz.launch.py

# стоп
pkill -f "ros2 launch"; pkill -f rviz2; pkill -f "gz sim"

# диагностика
ros2 node list | grep move_group
ros2 topic hz /joint_states
ros2 run tf2_ros tf2_echo world link1
```

14. Перенос проекта на новую машину (из готового репозитория)

14.1 Предусловия

Ubuntu 24.04 + базовый ROS 2 Kilted + интернет.

14.2 Зависимости

Раздел 1 целиком (git хранит только исходники; без `ros-kilted-ros2-control` сборка упадёт на `find_package(controller_manager_msgs)`).

14.3 Клонирование

```
mkdir -p ~/ros2k_ws/src
cd ~/ros2k_ws/src
git clone https://github.com/TolyaDenisenko/ros2kilt.git
cd ~/ros2k_ws
rm -rf ~/ros2k_ws/build ~/ros2k_ws/install ~/ros2k_ws/log # если попали в git
```

14.4–14.6 Окружение → stub-пакеты → сборка

Разделы 3, 5, 6 без изменений. Правки №1/№2 в репозитории уже применены.

14.7 Проверка сборки

```
ros2 pkg list | grep -E "xarm|moveit"
ls install/xarm_moveit_config/share/xarm_moveit_config/launch/xarm6_moveit_fake.launch.py
```

14.8 Запуск и проверки

Разделы 8–9. Чеклист: арт без ошибок; `/opt/ros/kilted` в `CMAKEPREFIXPATH`; stub-пакеты созданы; colcon без Failed; `/move_group` в node list; `/joint_states` > 50 Hz; RViz Status Ok + Plan/Execute работают.

15. Сменные grippers (end-effectors)

15.1 Типы и расположение

Gripper	URDF	MoveIt-конфиг	Аргумент	Сустав(ы)
xArm Gripper (XFG)	<code>xarm_description/urdf/gripper/xarm_gripper*.xacro</code>	<code>config/xarm_gripper</code>	<code>add_gripper:=true</code>	<code>drive_joint</code>
Lite Gripper	<code>.../gripper/uflite_gripper*.xacro</code>	<code>config/lite_gripper</code>	<code>add_gripper:=true</code> + <code>robot_type:=lite</code>	<code>drive_joint</code>
BIO Gripper	<code>.../gripper/bio_gripper*.xacro</code>	<code>config/bio_gripper</code>	<code>add_bio_gripper:=true</code>	<code>left_finger_joint</code> , <code>right_finger_joint</code>
Vacuum Gripper	<code>xarm_description/urdf/vacuum_gripper/</code>	—	<code>add_vacuum_gripper:=true</code>	нет суставов

15.2 Правила совместимости

- `add_gripper` и `add_vacuum_gripper` взаимоисключающие (для вакуума `add_gripper:=false`).
- `add_bio_gripper` приоритетнее `add_vacuum_gripper`.
- `add_gripper:=true` при `robot_type:=xarm` поднимает контроллер `xarm_gripper_traj_controller`

и planning-группу гриппера в MoveIt.

15.3 Команды запуска (fake)

```
ros2 launch xarm_moveit_config xarm6_moveit_fake.launch.py add_gripper:=true #
  xArm Gripper
ros2 launch xarm_moveit_config xarm6_moveit_fake.launch.py add_vacuum_gripper:=true
# вакуум
ros2 launch xarm_moveit_config xarm6_moveit_fake.launch.py add_bio_gripper:=true
# BIO
ros2 launch xarm_moveit_config lite6_moveit_fake.launch.py add_gripper:=true
# Lite
```

15.4 Проверка и диапазоны

```
ros2 topic echo /joint_states --once | grep -E "drive_joint|finger_joint"
ros2 control list_controllers | grep gripper
grep -A4 "drive_joint" \
    ~/ros2k_ws/src/xarm_ros2/xarm_moveit_config/config/xarm_gripper/joint_limits.yaml
```

15.5 Управление из Python

```
import moveit_commander

gripper = moveit_commander.MoveGroupCommander("xarm_gripper")
gripper.set_joint_value_target({"drive_joint": 0.0}) # раскрыт
gripper.go(wait=True)
```

15.6 Нюансы Kilted-порта

Пересборка не нужна; старая шапка OMPL была только в общей `moveit_configs/ompl_planning.yaml` (исправлена, раздел 7). Файлы групп начинаются с имени группы — шапки не содержат.

15.7 Шпаргалка по gripper

```
ros2 launch xarm_moveit_config xarm6_moveit_fake.launch.py --show-args | grep -i gripper
r
ls ~/ros2k_ws/src/xarm_ros2/xarm_description/urdf/gripper/
```

15.8 Gripper в fake-режиме (RViz + MoveIt, без Gazebo)

```
ros2 launch xarm_moveit_config xarm6_moveit_fake.launch.py add_gripper:=true
```

Проверка:

```
ros2 control list_controllers | grep gripper          # xarm_gripper_traj_controller active
ros2 topic echo /joint_states --once | grep -A2 drive_joint
```

RViz: **MotionPlanning** → **Planning** → **Planning Group** → `xarm_gripper` → слайдер `drive_joint` → Plan & Execute. Python-управление — как в 15.5.

15.9 Gripper в Gazebo (физическая симуляция)

Требует в `xarm_gz_bringup/launch/xarm6_gz.launch.py` условный спавнер контроллера гриппера:

```
from launch.conditions import IfCondition

...

gripper_traj_controller_spawner = Node(
    package='controller_manager',
    executable='spawner',
    arguments=['xarm_gripper_traj_controller', '--controller-manager', '/controller_manager'],
    parameters=[{'use_sim_time': True}],
    output='screen',
    condition=IfCondition(add_gripper),
)

# и добавить его в actions таймера controllers_delay
```

Запуск и проверка:

```
ros2 launch xarm_gz_bringup xarm6_gz.launch.py add_gripper:=true
# после «You can start planning now!»:
ros2 control list_controllers | grep gripper
ros2 topic echo /joint_states --once | grep drive_joint
gz model --list
```

Пальцы должны физически сомкнуться в окне Gazebo (Planning Group `xarm_gripper` в RViz).

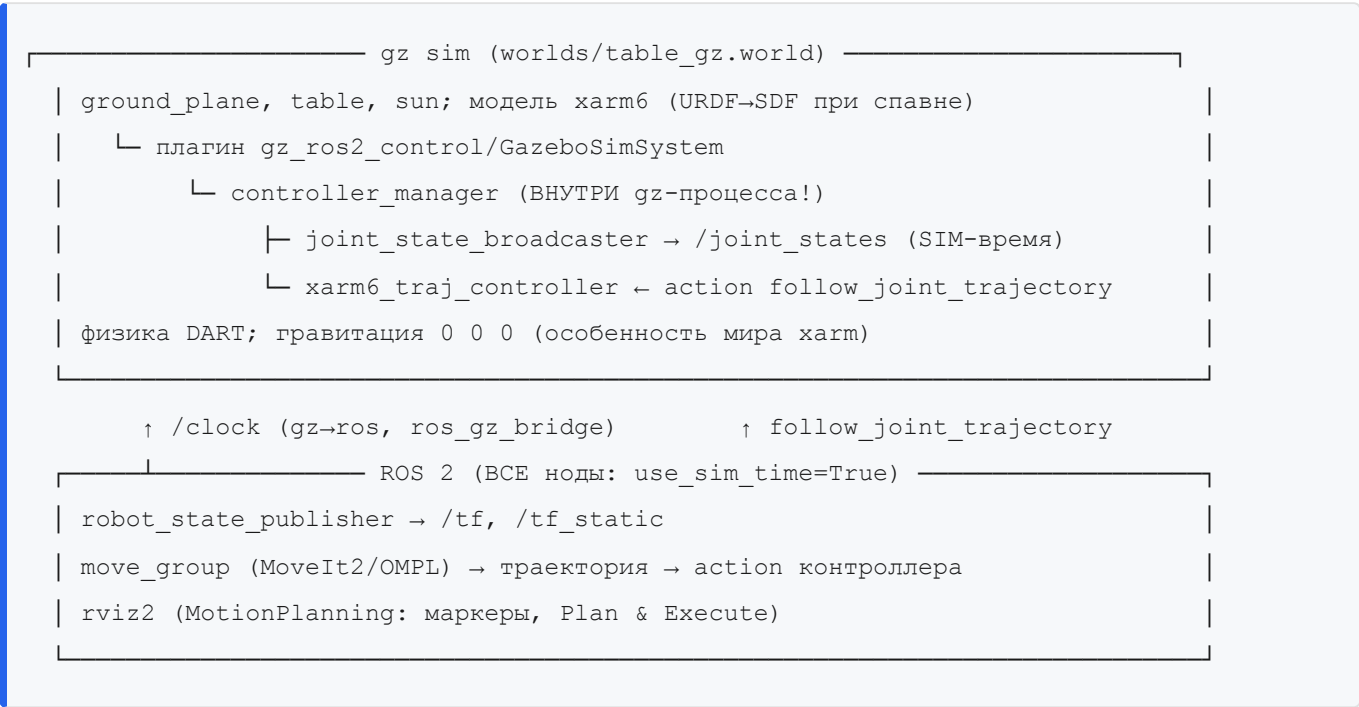
⚠ Статус: процедура подготовлена; контакты и *mitic*-суставы пальцев в *gz sim* полностью не прогонялись. Если гриппер не сжимает предмет — смотреть лог *gz* на ошибки *mitic/joint*.

16. Симуляция в Gazebo (gz sim): порт на Kilted

16.1 Почему не xarm_gazebo

`xarm_gazebo` написан под Gazebo Classic (EOL с 01.2025, в Kilted отсутствует). Создан пакет `xarm_gz_bringup`: `gz sim 9 (Ionic)` + `ros_gz_sim` + `gz_ros2_control`. URDF `xarm_ros2` параметризован по `ros2_control_plugin` — подменяем на `gz_ros2_control/GazeboSimSystem` (ветка уже подготовлена апстримом).

16.2 Архитектура



16.3 Состав xarmgzbringup и тайминги

Файл	Назначение
<code>launch/xarm6_gz.launch.py</code>	gz sim, спавн робота, спавнеры, MoveIt, мост /clock
<code>worlds/table_gz.world</code>	копия мира из xarm_gazebo (SDF нового Gazebo)
<code>config/gz_bridge.yaml</code>	<code>/clock</code> : gz.msgs.Clock → rosgraph_msgs/msg/Clock

Тайминги: 0 с — gz sim; 3 с — спавн робота; 5 с — спавнеры контроллеров; **12 с — MoveIt+RViz** (сдвиг обязателен: `/clock` уже течёт, move_group рождается на сим-времени без «нулевого» окна).

16.4 Запуск и протокол готовности

```
killall -f "ros2 launch"; killall -f rviz2; killall -f "gz sim"; sleep 2
cd ~/ros2k_ws && source install/setup.bash
ros2 launch xarm_gz_bringup xarm6_gz.launch.py 2>&1 | tee /tmp/gz.log
```

Ждём по признакам:

```
grep -m1 "You can start planning now" /tmp/gz.log      # move_group готов
ros2 param get /move_group use_sim_time                # True
ros2 control list_controllers                          # оба active
```

Только затем Plan & Execute в RViz.

16.5 Проверки и ручная траектория (минуя MoveIt)

```
ros2 control list_controllers
ros2 action list | grep follow_joint_trajectory
timeout 5 ros2 topic hz /joint_states

ros2 action send_goal /xarm6_traj_controller/follow_joint_trajectory \
  control_msgs/action/FollowJointTrajectory \
  "{trajectory: {joint_names: [joint1, joint2, joint3, joint4, joint5, joint6], point
s: [{positions: [0.3, -0.5, 0.1, 0.0, 0.5, 0.0], time_from_start: {sec: 3}}]}}"
```

⚠ Предел `joint3 = +0.19198` рад: выше — `Command out of limits` (робот упрётся в предел).

16.6 Урок порта: сим-время (usesimtime)

Симптом: `Didn't receive robot state ... latest received state has time 3.07`, `Failed to validate trajectory ... within 1s`, `CONTROL_FAILED` при живых контроллерах. **Причина:** метки сим-времени из симуляции против настенных часов нод (~1.79e9). **Решение:** `use_sim_time: True` в `parameters=[...]` в момент рождения каждой ноды: `movegroup`, `rviz2`, `statictf` (форк `_robot_moveit_common.launch.py`), `robotstatepublisher` (форк `_robot_description.launch.py`), спавнеры, `create`, `parameter_bridge`. **НЕ повторять:** `SetParameter(...)` не пробивает чужие `include`-ы (`OpaqueFunction`); runtime `ros2 param set` ненадёжен во время инициализации; таймер-фиксер — гонка. **Индикатор рассинхрона:** `tf2_buffer: Detected jump back in time, time diff: 179...`.

16.7 Known issues (Gazebo)

Явление	Статус
<code>Could not find the planner configuration ...</code>	косметика OMPL
<code>Ogre material scripts</code> + серый робот в gz	косметика SDF-конвертации
Первые ~10 с: Execute может отвалиться	warm-up <code>/clock</code> , ждать протокол готовности
Поза спавна <code>-x 0 -y -0.84 -z 1.0</code>	подбирать z под стол (~0.75–0.8)
Гриппер в gz: контакты/mimic	см. 15.9, следующий слой отладки

16.8 Перенос и git

`xarm_gz_bringup` + правки форка закоммичены. На новой машине: раздел 1 → клонирование → stub-пакеты (5) → сборка (6) → `ros2 launch xarm_gz_bringup xarm6_gz.launch.py`.

Приложение А. Промт для продолжения работы с ИИ-ассистентом

Ты – эксперт по ROS 2, MoveIt 2, ros2_control и Gazebo (gz sim). Продолжаем проект: xarm_ros2 (UFactory xArm6), портированный на ROS 2 Kilted + физическая симуляция.

Контекст:

- Ubuntu 24.04, ROS 2 Kilted, MoveIt 2.14, ros2_control 5.x, gz sim 9 (Ionic), Python 3.12.
- Workspace ~/ros2k_ws; исходники ~/ros2k_ws/src/xarm_ros2; свой пакет ~/ros2k_ws/src/xarm_gz_bringup.
- Документация: ~/ros2k_ws/XARM_ROS2_KILTIED.md (разделы 1-16; читай первым делом).
- Репозитории: github.com/TolyaDenisenko/ros2kilt и .../ros2k_ws.
- Применённые правки: удалён include visibility_macros.hpp; ompl_planning.yaml переписан на Kilted-формат; use_sim_time=True у move_group/rviz2/static_tf/robot_state_publisher;
- stub-пакет xarm_gazebo в install/.
- Режимы: fake – xarm_moveit_config xarm6_moveit_fake.launch.py; Gazebo – xarm_gz_bringup xarm6_gz.launch.py (протокол готовности: раздел 16.4).
- Grippers: xArm/Lite/BIO/Vacuum, аргументы add_gripper/add_bio_gripper/add_vacuum_gripper (раздел 15; в Gazebo нужен спавнер xarm_gripper_traj_controller, 15.9).
- Не собираются: xarm_gazebo, realsense_gazebo_plugin, d435i_xarm_setup.

Правила ответа:

- Отвечай на русском, пошагово, по одному блоку команд, жди мой вывод.
- Только команды, актуальные для Kilted/gz sim 9; при неуверенности – сначала диагностика.
- Разделяй warning и error; не пересобирай работающее.
- Помни про sim time: все новые ноды рождать с use_sim_time=True (урок 16.6).
- После rm -rf install/ build/ – заново stub-пакеты (раздел 5).

Текущая задача: <ОПИШИ ЗАДАЧУ: гриппер с контактами / тележка + рука / своя нода / новая ошибка>